

Wprowadzenie

Niniejszy raport opisuje realizację projektu polegającego na implementacji probabilistycznego wariantu gry w kółko i krzyżyk — **Tic-tac-doh** — z wykorzystaniem biblioteki EasyAI, a także na przeprowadzeniu eksperymentów porównujących wydajność różnych algorytmów przeszukiwania drzewa gry.

Opis gry

Tic-tac-doh to probabilistyczny wariant klasycznej gry w kółko i krzyżyk (3×3). Reguły są identyczne z klasyczną wersją, z jednym wyjątkiem:

Z **20% prawdopodobieństwem** ruch gracza się nie udaje — na planszy nie pozostaje żaden ślad, a przeciwnik wykonuje ruch ponownie.

To wprowadza element losowości do gry, która w klasycznej formie jest w pełni deterministyczna. W konsekwencji:

- Gra, która w wariacie deterministycznym zawsze kończy się remisem (przy optymalnej grze obu stron), w wariacie probabilistycznym może zakończyć się wygraną jednego z graczy.
- Algorytmy AI zaprojektowane dla gier deterministycznych (np. Negamax) nie modelują tego ryzyka, co może wpływać na jakość ich decyzji.

Implementacja gry

Gra Tic-tac-doh została zaimplementowana w pliku `tictac.py` jako klasa `TicTacDoh` dziedzicząca po `TwoPlayerGame` z biblioteki EasyAI. Kluczowe elementy implementacji:

- **Plansza:** tablica 9 pól, wartości `0` (puste), `1` (gracz O), `2` (gracz X).
- **Parametr `probabilistic`:** przełącznik między wariantem deterministycznym (`False`) a probabilistycznym (`True`).
- **Mechanizm `failure`:** w metodzie `make_move()`, przy `probabilistic=True`, losowane jest zdarzenie z prawdopodobieństwem 20%. Jeżeli ruch się nie uda, plansza pozostaje bez zmian, a kolejka przechodzi do przeciwnika.
- **Separacja losowości od symulacji AI:** flaga `_apply_failure` jest aktywna **wyłącznie** podczas faktycznego wykonania ruchu, a nie podczas przeszukiwania drzewa gry przez algorytm AI. Dzięki temu symulacja Negamax jest w pełni deterministyczna — losowość odzwierciedlona jest jedynie w fizycznym wykonaniu ruchu.

Funkcja oceny (`scoring`) zwraca `-$100$` w przypadku przegranej i `$0$` w przeciwnym razie.

Porównywane algorytmy

W ramach projektu zaimplementowano i porównano cztery algorytmy przeszukiwania drzewa gry:

Negamax z odcięciem alfa-beta

Standardowy algorytm z biblioteki EasyAI. Negamax to wariant algorytmu Minimax, gdzie wartość pozycji jest negowana przy zmianie gracza, eliminując potrzebę osobnych faz minimalizacji i maksymalizacji. Odcięcie alfa-beta przyspiesza przeszukiwanie poprzez eliminację gałęzi drzewa, które nie mogą wpłynąć na wynik.

Negamax bez odcięcia alfa-beta

Własna implementacja (plik `negamax_no_ab.py`), zgodna z interfejsem EasyAI. Przeszukuje **pełne drzewo gry** bez jakichkolwiek odcieć. Daje identyczne wyniki jak Negamax z alfa-beta, ale jest znacznie wolniejsza, ponieważ eksponuje każdy węzeł.

SSS*

Algorytm SSS* z biblioteki EasyAI. Jest to algorytm best-first search, który przeszukuje drzewo gry w kolejności „najbardziej obiecujących” gałęzi. Teoretycznie odwiedza mniejszą liczbę węzłów niż alfa-beta, ale wymaga dodatkowej pamięci.

Expecti-Minimax z odcięciem alfa-beta

Własna implementacja (plik `expectiminimax.py`), zaprojektowana specjalnie dla gier z elementem losowości. W odróżnieniu od standardowego Negamax, Expecti-Minimax **modeluje węzły losowe (chance nodes)** w drzewie gry.

Dla każdego ruchu algorytm oblicza wartość oczekiwaną:

$$V(\text{ruch}) = (1-p) \cdot V(\text{sukces}) + p \cdot V(\text{porażka})$$

gdzie $p = 0,2$ to prawdopodobieństwo nieudanego ruchu. W przypadku porażki plansza pozostaje bez zmian, a kolej przechodzi do przeciwnika.

Odcięcie alfa-beta na węzłach losowych wykorzystuje technikę **Star1 pruning**, która ogranicza przeszukiwanie na podstawie pesymistycznych i optymistycznych oszacowań wartości oczekiwanej. Dzięki temu algorytm nie musi zawsze eksplorować obu gałęzi (sukces/porażka) dla każdego ruchu — jeśli znana jest wartość jednej gałęzi, można zawęzić granice alfa-beta dla drugiej.

Eksperymenty

Eksperymenty polegały na wielokrotnym rozgrywaniu partii **AI vs AI**, z wymianą gracza rozpoczynającego co partię, w celu zbadania:

1. Wpływu probabilistyczności na wyniki gry.
2. Wpływu głębokości przeszukiwania na jakość gry.
3. Różnic wydajnościowych (czas obliczenia ruchu) między algorytmami.
4. Różnic w skuteczności algorytmu uwzględniającego losowość (Expecti-Minimax) względem algorytmów deterministycznych.

Wszystkie eksperymenty uruchomiono z ziarnem generatora losowego (`random.seed(42)`) dla powtarzalności wyników.

Wpływ głębokości Negamax na wyniki gry

Pierwszy eksperyment porównuje algorytm Negamax (z odcięciem alfa-beta) przy dwóch głębokościach przeszukiwania: **4 i 8**, na obu wariantach gry. Rozegrano **100 partii** na konfigurację.

Głębokość	Wariant	Gracz 1	Gracz 2	Remisy
4	Deterministyczny	0	0	100
4	Probabilistyczny	42	24	34
8	Deterministyczny	0	0	100
8	Probabilistyczny	35	16	49

Obserwacje:

- W wariantcie **deterministycznym** wszystkie partie kończą się remisem, niezależnie od głębokości. Tic-tac-toe jest grą o pełnej informacji z optymalnym rozwiązaniem remisowym, a głębokość 4 jest już wystarczająca, by AI grało bezbłędnie.
- W wariantcie **probabilistycznym** losowość powoduje, że wiele partii kończy się wygraną jednego z graczy. Gracz 1 (rozpoczynający) wygrywa częściej, co jest przewagą wynikającą z inicjatywy.
- Przy **głębokości 8** Negamax gra lepiej niż przy głębokości 4 w wariantcie probabilistycznym — więcej remisów (49 vs 34), co oznacza, że AI rzadziej popełnia błędy, nawet gdy ruch się nie uda. Głębsze przeszukiwanie pozwala lepiej kompensować nieudane ruchy.
- W obu wariantach probabilistycznych procent nieudanych ruchów oscyluje wokół oczekiwanych 20% (18,0% i 18,1%), co potwierdza poprawność implementacji.

Porównanie algorytmów: wydajność i czas

Drugi eksperyment porównuje **cztery algorytmy** na obu wariantach gry, z głębokościami przeszukiwania **2 i 6**. Rozegrano **50 partii** na konfigurację. Wyniki eksperymentów dla algorytmów Negamax (z α - β i bez), SSS* oraz ExpectiMinimax zestawiono w poniższych tabelach.

Głębokość 2

Algorytm	Wariant	G1	G2	Rem	Śr. czas/ruch
Negamax (α - β)	Deter.	50	0	0	93 μ s
Negamax (α - β)	Prob.	36	12	2	93 μ s
Negamax (bez α - β)	Deter.	50	0	0	204 μ s
Negamax (bez α - β)	Prob.	36	12	2	207 μ s
SSS*	Deter.	50	0	0	124 μ s
SSS*	Prob.	36	12	2	120 μ s
ExpectiMinimax (α - β)	Deter.	50	0	0	522 μ s
ExpectiMinimax (α - β)	Prob.	36	12	2	482 μ s

Głębokość 6

Algorytm	Wariant	G1	G2	Rem	Śr. czas/ruch
Negamax (α - β)	Deter.	0	0	50	2,59 ms
Negamax (α - β)	Prob.	20	10	20	3,34 ms
Negamax (bez α - β)	Deter.	0	0	50	61,02 ms
Negamax (bez α - β)	Prob.	17	14	19	72,53 ms
SSS*	Deter.	0	0	50	3,93 ms
SSS*	Prob.	17	14	19	4,38 ms
ExpectiMinimax (α - β)	Deter.	0	0	50	1,82 s
ExpectiMinimax (α - β)	Prob.	19	10	21	2,98 s

Analiza wydajności algorytmów

Wpływ odcięcia alfa-beta

Porównanie Negamax z i bez odcięcia alfa-beta wyraźnie pokazuje wartość tej optymalizacji:

Głębokość	Bez α - β	Z α - β	Przyspieszenie
2	204 μ s	93 μ s	2,2×
6	61,02 ms	2,59 ms	23,6×

Przyspieszenie rośnie wykładniczo z głębokością. Przy głębokości 6 odcięcie alfa-beta jest **ponad 23-krotnie szybsze**, ponieważ eliminuje znaczną część drzewa gry. Jest to zgodne z teorią — odcięcie alfa-beta w najlepszym przypadku redukuje liczbę odwiedzanych węzłów z $O(b^d)$ do $O(b^{\lceil d/2 \rceil})$, gdzie b to współczynnik rozgałęzienia, a d to głębokość.

SSS* vs Negamax (α - β)

SSS* jest nieznacznie wolniejszy od Negamax z alfa-beta:

Głębokość	Negamax (α - β)	SSS*	Stosunek
2	93 μ s	124 μ s	1,3×
6	2,59 ms	3,93 ms	1,5×

SSS* wymaga dodatkowej pamięci do utrzymania listy otwartych węzłów, co wprowadza narzut. W praktyce, dla Tic-tac-toh, nie oferuje przewagi nad Negamax z alfa-beta.

ExpectiMinimax vs Negamax (α - β)

ExpectiMinimax jest znacząco wolniejszy, ponieważ dla każdego ruchu musi zbadać dwa scenariusze (sukces + porażka):

Głębokość	Negamax (α - β)	ExpectiMinimax	Stosunek
2	93 μ s	522 μ s	5,6×
6	2,59 ms	1,82 s	700×

Dramatyczny wzrost kosztów przy głębokości 6 wynika z tego, że **każdy węzeł** generuje podwójne rozgałęzienie (sukces/porażka), co efektywnie podwaja współczynnik rozgałęzienia drzewa. Przy głębokości $d=6$ oznacza to potencjalnie $2^6 = 64$ -krotny wzrost liczby odwiedzanych węzłów, chociaż pruning Star1 częściowo kompensuje ten efekt.

Analiza jakości decyzji

Wariant deterministyczny

W wariancie deterministycznym przy głębokości 6 wszystkie algorytmy osiągają taki sam wynik — **100% remisów**. Oznacza to, że głębokość 6 jest wystarczająca, aby każdy z algorytmów grał optymalnie.

Przy głębokości 2 wszystkie algorytmy dają identyczny wynik (Gracz 1 wygrywa 50 gier na 50), co sugeruje, że Gracz 2 gra suboptymalnie z powodu zbyt płytkiego przeszukiwania.

Wariant probabilistyczny

W wariancie probabilistycznym pojawiają się różnice:

Algorytm (głęb. 6)	Gracz 1	Gracz 2	Remisy
Negamax (α - β)	20	10	20
Negamax (bez α - β)	17	14	19
SSS*	17	14	19
ExpectiMinimax (α - β)	19	10	21

ExpectiMinimax (21 remisów) osiąga porównywalne wyniki do Negamax z alfa-beta (20 remisów), oba lepsze od Negamax bez alfa-beta i SSS* (po 19 remisów). Więcej remisów oznacza lepszą obronę — AI potrafi zminimalizować wpływ losowości na wynik.

Warto zauważyć, że **Negamax bez alfa-beta** i **SSS*** dają identyczne wyniki pod względem wygranych/przegranych/remisów, co jest zgodne z oczekiwaniami — oba przeszukują drzewo w pełni (lub zbliżonej do pełnej) i podejmują identyczne decyzje.

Napotkane problemy

1. **Separacja losowości od symulacji AI** — największym wyzwaniem przy implementacji Tic-tac-doh było zapewnienie, że losowość (20% nieudanych ruchów) nie wpływa na przeszukiwanie drzewa gry przez algorytmy AI. Algorytmy takie jak Negamax wielokrotnie wywołują `make_move / unmake_move` podczas symulacji. Gdyby losowość była aktywna w tych wywołaniach, symulacja byłaby niedeterministyczna, co prowadziłoby do nieprzewidywalnego zachowania AI. Problem

rozwiązano za pomocą flagi `_apply_failure`, która jest aktywna wyłącznie przy faktycznym wykonaniu ruchu.

2. **Wydajność Expecti-Minimax** — algorytm jest $\sim 700\times$ wolniejszy od Negamax przy głębokości 6 (2,98 s vs 3,34 ms na ruch). Podwójne rozgałęzienie na każdym węźle (sukces/porażka) dramatycznie zwiększa liczbę odwiedzanych węzłów. Mimo zastosowania pruningu Star1, narzut ten jest nieunikniony. W praktyce ogranicza to głębokość przeszukiwania, do jakiej ExpectiMinimax może być stosowany w rozsądnym czasie.
3. **Zgodność interfejsów** — własne implementacje algorytmów (`NegamaxNoAB`, `ExpectiMinimax`) musiały być zgodne z interfejsem `AI_Player` z biblioteki EasyAI, co wymagało implementacji metody `__call__(game) -> move` ustawiającej atrybut `game.ai_move`.

Podsumowanie

Projekt obejmował implementację probabilistycznej gry Tic-tac-doh oraz porównanie czterech algorytmów przeszukiwania drzewa gry: Negamax z odcięciem alfa-beta, Negamax bez odcięcia, SSS* oraz Expecti-Minimax z odcięciem alfa-beta.

Kluczowe wnioski:

- **Odcięcie alfa-beta** jest krytyczną optymalizacją — przyspiesza przeszukiwanie od $2\times$ (głębokość 2) do ponad $23\times$ (głębokość 6), bez wpływu na jakość decyzji.
- **SSS*** nie oferuje istotnej przewagi nad Negamax z alfa-beta w kontekście Tic-tac-doh — jest nieco wolniejszy i daje identyczne wyniki.
- **Głębsze przeszukiwanie** poprawia wyniki w wariancie probabilistycznym — AI lepiej kompensuje losowe porażki ruchów.
- **Expecti-Minimax** jako jedyny algorytm explicite modeluje losowość w drzewie gry. Osiąga nieznacznie lepsze wyniki w wariancie probabilistycznym (więcej remisów), ale kosztem dramatycznie większego czasu obliczeń ($\sim 700\times$ wolniejszy od Negamax z alfa-beta przy głębokości 6). Wynika to z podwójnego rozgałęzienia na każdym węźle (scenariusze sukces/porażka), co efektywnie powiększa drzewo gry.
- W kontekście Tic-tac-doh, ze względu na stosunkowo płytke drzewo gry (maks. 9 ruchów), różnice w jakości decyzji między Negamax ($\alpha\text{-}\beta$) a Expecti-Minimax nie są znaczące. Algorytm Expecti-Minimax miałby większy wpływ w grach z głębszym drzewem i wyższym prawdopodobieństwem losowych zdarzeń.